

A Comparative Empirical Analysis of Sorting Algorithms

Under Varied Input Distributions

Daniel Pantea

Faculty of Informatics, Department of Computer Science

West University of Timișoara

daniel.pantea06@e-uvv.ro

May 10, 2026

Abstract

Almost every software system depends on sorting at some point — whether it is ranking search results, scheduling tasks, or looking up records in a database. Even though sorting has been studied for a long time, choosing the right algorithm for a specific situation is still not simple. The reason is that theoretical complexity alone does not tell the full story. How fast an algorithm runs also depends on the shape of the input data. This paper studies that question for six well-known sorting algorithms: Bubble Sort, Insertion Sort, Selection Sort, Quick Sort, Merge Sort, and Radix Sort.

Instead of testing only fully random or fully reversed arrays, we also test *partially sorted* arrays. These are arrays that start in order but then have a controlled number of random swaps applied. We test seven different input sizes, from 10 to 1 000 000 elements, and repeat each test many times to get stable timing results. We measure how long each sort takes in milliseconds.

The results show some surprising findings. Insertion Sort, which is usually seen as a slow $O(n^2)$ algorithm, can be as fast as Quick Sort when the data is nearly sorted. Radix Sort is the fastest at large sizes on random data, finishing one million elements in under 100 ms. Merge Sort is the most consistent — its speed barely changes no matter what kind of input it receives. These findings give clear, practical advice on which algorithm to choose depending on the situation.

Contents

1	Introduction	3
1.1	Organization of the Paper	3
2	Problem Formulation and Algorithmic Approaches	3
2.1	Problem Definition	3
2.2	Algorithm Descriptions and Complexity	3
2.3	Summary of Theoretical Complexity	4
3	Implementation and Architecture	4
3.1	Architecture Overview	4
3.2	Data Structures	5
3.3	Partial Sort Generator	5
3.4	Median-of-Three Quick Sort	5
3.5	User Manual	5
4	Experimental Evaluation	6
4.1	Experimental Setup	6
4.2	Results: Small Arrays ($n \leq 100$)	6
4.3	Results: Medium Arrays ($n = 1000$)	7
4.4	Results: Large Arrays ($n = 10\,000$ and $n = 100\,000$)	7
4.5	Results: Very Large Arrays ($n = 1\,000\,000$)	7
4.6	Visual Analysis	8
4.7	Key Observations	8
5	Related Work	9
6	Conclusions and Future Work	10
6.1	Summary	10
6.2	Problems Encountered	10
6.3	Future Work	11
A	Pseudocode	11
A.1	Partially sorted array generator	12
A.2	Median-of-three pivot selection	12

1 Introduction

1.1 Organization of the Paper

The background theory in this paper is based on well-known sources listed in Section 5. The original work done by the author includes: (1) designing and writing the C benchmarking program, including the function that creates partially sorted arrays with a chosen level of disorder; (2) running tests across seven input sizes, six algorithms, and seven input types; and (3) studying how the order of the input affects the speed of each algorithm, with a focus on Insertion Sort and Bubble Sort on partially sorted data.

Section 2 gives formal definitions and explains the complexity of each algorithm. Section 3 describes how the program was built and how the data structures work. Section 4 explains the test setup and shows the results. Section 5 reviews related work done by others. Section 6 summarizes the main findings and suggests ideas for future work. Section 6.3 lists all sources used.

2 Problem Formulation and Algorithmic Approaches

2.1 Problem Definition

Definition 2.1 (Sorting Problem). [3, 2] Given a sequence $A = (a_1, a_2, \dots, a_n)$ of n elements drawn from a totally ordered set $(S, \leq)$, produce a permutation $A' = (a_{\pi(1)}, a_{\pi(2)}, \dots, a_{\pi(n)})$ such that $a_{\pi(i)} \leq a_{\pi(i+1)}$ for all $1 \leq i < n$.

In this paper, $S = \mathbb{Z}$ and all comparisons are done on 32-bit signed integers.

Definition 2.2 (Inversion Count). [2, 3] For a sequence A of length n , the inversion count is $\text{inv}(A) = |\{(i, j) : i < j \text{ and } a_i > a_j\}|$. A fully sorted array has $\text{inv}(A) = 0$; a fully reversed array has $\text{inv}(A) = \binom{n}{2}$.

The inversion count is a way to measure how unsorted an array is. Algorithms that directly depend on this count (such as Insertion Sort) will run faster when the count is low.

2.2 Algorithm Descriptions and Complexity

Bubble Sort. Goes through the array repeatedly, swapping two neighbouring elements if they are in the wrong order. It stops early if a full pass made no swaps. Best case is $\Theta(n)$ (already sorted), worst case is $\Theta(n^2)$ (fully reversed). The total number of swaps equals $\text{inv}(A)$.

Insertion Sort. Keeps a sorted section at the start of the array, and places each new element into the correct position by shifting elements one step to the right. Its running time is $\Theta(n + \text{inv}(A))$ — this means it is very fast on nearly sorted data and slow on reversed data. It also accesses memory in order, which is good for cache performance.

Selection Sort. Finds the smallest element in the unsorted part of the array and puts it at the front. The number of comparisons is always $\Theta(n^2)$, no matter what the input looks like. Only the number of swaps changes. This means its speed is not affected by the order of the input.

Quick Sort. Splits the array around a chosen pivot element and sorts each part separately using recursion. This implementation uses median-of-three pivot selection:

Definition 2.3 (Median-of-Three Pivot). [5] Given indices ℓ , $m = \lfloor (\ell + r)/2 \rfloor$, and r , sort $\{A[\ell], A[m], A[r]\}$ in place and use $A[r]$ (the median) as the pivot.

Theorem 2.1. Quick Sort with median-of-three achieves an expected $O(n \log n)$ number of comparisons on random input and avoids the $O(n^2)$ worst case that happens with a naive pivot on already-sorted or reversed arrays.

Merge Sort. Splits the array into two halves, sorts each half, and then joins them back together in order. Its time complexity is $\Theta(n \log n)$ in all cases. It needs $\Theta(n)$ extra memory for the merge step. Because the merge step does not depend on the order of the input, Merge Sort behaves the same way no matter what data it receives.

Radix Sort. Does not use comparisons. Instead, it sorts integers digit by digit, starting from the least significant digit, using a stable counting sort at each step. With d digits in base b :

Proposition 2.1. Radix Sort runs in $\Theta(d(n + b))$ time and uses $\Theta(n + b)$ extra memory. For integers of fixed size in base 10, $d = \lfloor \log_{10}(\max A) \rfloor + 1$, which means it behaves like $O(n)$ for bounded integer values.

2.3 Summary of Theoretical Complexity

Table 1 summarizes the time and space complexity of the six algorithms discussed in this section.

Table 1: Theoretical time complexity of the six algorithms.

Algorithm	Best Case	Average Case	Worst Case	Space
Bubble Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$
Quick Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)^*$	$O(\log n)$
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$
Radix Sort	$\Theta(dn)$	$\Theta(dn)$	$\Theta(dn)$	$O(n + b)$

*Mitigated by median-of-three pivot in this implementation.

3 Implementation and Architecture

3.1 Architecture Overview

The benchmark is written as a single C file, compiled with GCC. The code is split into four clear parts:

1. **Data generation:** Functions `generateIntArray`, `generateReverseSortedIntArray`, and `generatePartiallySortedIntArray` create the test arrays.

2. **Sorting functions:** One function per algorithm, each working directly on the input array.
3. **Timing loop:** `analyzeSortAlg` runs each algorithm many times, using `memcpy` to reset the array to its original state before each run.
4. **Output:** Results are saved to both the terminal and a CSV file.

3.2 Data Structures

All arrays are stored on the heap using `malloc` and freed with `free`. The timing loop keeps a *master* copy of the original array (which is never changed) and a *working* copy that is reset before each timed run. This makes sure every run starts with exactly the same data. Time is measured using the `clock()` function, which gives CPU time in milliseconds.

3.3 Partial Sort Generator

One of the new features of this benchmark is the partially sorted input generator. It builds an ordered array and then makes a controlled number of random swaps to introduce disorder. The number of swaps is n/factor , so a smaller factor creates more disorder and a larger factor keeps the array closer to sorted. Short pseudocode for this generator appears in Appendix A.

3.4 Median-of-Three Quick Sort

The pivot selection function works as follows. It looks at the first, middle, and last elements of the current subarray, sorts those three values using simple comparisons, and places the middle value at the end to use as the pivot. This helps avoid the worst case on already-sorted or reversed arrays. A short pseudocode sketch is available in Appendix A.

3.5 User Manual

This section gives a brief and simple guide to compile and run the benchmark.

Prerequisites

The benchmark requires only a C99-compatible compiler and the standard C library. No extra libraries are needed. The tools used for the original tests were:

- **GCC** (version 7 or later), or another C99 compiler such as MSVC or Clang.
- **CMake** (version 3.10 or later), which was used to produce the release build in the original study.
- A terminal or command prompt where the program can write the output file `results1.csv`.

Building and Running

Compile the program in release mode and then run the executable. On Windows, the output is typically `sorting_algorithms.exe`. The benchmark runs automatically with no command-line arguments and writes the results to a CSV file.

Configuration

All experimental settings are controlled from the `main()` function. You can change the input sizes, repetition counts, and output file name in one place. The default benchmark is set up so that small inputs run many times for accuracy and large inputs run once to keep total execution time reasonable.

Output Format

The program writes results both to the terminal and to `results1.csv`. Each row contains the number of elements, the number of runs, the algorithm name, the input variant, the average elapsed time in milliseconds, and a short summary message.

Interpreting the Results

The output is designed to be easy to read. For example, one line reports how long Quick Sort took on a random array of 1 000 elements. The benchmark uses more repetitions for small sizes to ensure the measured time is meaningful.

4 Experimental Evaluation

4.1 Experimental Setup

- **Platform:** Windows 10, compiled with GCC via CMake (Release mode).
- **Input sizes:** $n \in \{10, 50, 100, 1000, 10\,000, 100\,000, 1\,000\,000\}$.
- **Repetitions:** 1 000 000 runs for $n \leq 100$; 100 for $n = 1000$; 10 for $n = 10\,000$; 1 for $n \geq 100\,000$.
- **Input variants:** random, reversed, partially sorted at 10%, 30%, 50%, 70%, 90% factor.
- **Metric:** Average elapsed CPU time per sort, in milliseconds.

4.2 Results: Small Arrays ($n \leq 100$)

For small arrays, the cost of function calls and memory allocation is large compared to the actual sorting work. Merge Sort is the slowest at $n = 10$ and $n = 50$ because it creates and deletes helper arrays during each merge, which adds a lot of overhead for such a small input. Insertion Sort and Bubble Sort (with early exit) are close to Quick Sort in speed for sorted and nearly-sorted inputs.

Table 2: Elapsed time (ms) for $n = 100$, averaged over 1 000 000 runs per variant.

Algorithm	Random	Rev.	PS 10%	PS 30%	PS 50%	PS 70%	PS 90%
Bubble Sort	0.01863	0.02405	0.01141	0.00626	0.00461	0.00718	0.00688
Quick Sort	0.00283	0.00367	0.00285	0.00289	0.00331	0.00290	0.00278
Insertion Sort	0.00693	0.01247	0.00243	0.00109	0.00078	0.00043	0.00051
Selection Sort	0.01148	0.01098	0.01102	0.01051	0.01049	0.01053	0.01025
Merge Sort	0.01648	0.01540	0.01584	0.01602	0.01537	0.01515	0.01564
Radix Sort	0.00568	0.00385	0.00379	0.00387	0.00392	0.00389	0.00380

4.3 Results: Medium Arrays ($n = 1000$)

At $n = 1000$, the $O(n^2)$ algorithms start to clearly fall behind the $O(n \log n)$ ones. Quick Sort and Radix Sort are both around 0.07–0.09 ms on most inputs. Insertion Sort improves greatly on partially sorted data: on the 90% factor (almost sorted), it takes only 0.01 ms compared to 0.62 ms on random data — 62 times faster.

Table 3: Elapsed time (ms) for $n = 1000$, averaged over 100 runs.

Algorithm	Random	Rev.	PS 10%	PS 30%	PS 50%	PS 70%	PS 90%
Bubble Sort	1.860	2.380	1.290	1.050	1.040	0.990	1.100
Quick Sort	0.090	0.080	0.070	0.060	0.060	0.070	0.070
Insertion Sort	0.620	1.130	0.140	0.050	0.040	0.030	0.010
Selection Sort	1.080	1.030	1.050	1.020	1.050	1.040	1.000
Merge Sort	0.240	0.190	0.190	0.170	0.180	0.180	0.180
Radix Sort	0.070	0.060	0.050	0.060	0.050	0.050	0.060

4.4 Results: Large Arrays ($n = 10\,000$ and $n = 100\,000$)

At $n = 10\,000$, Bubble Sort takes 244 ms on random data, but only 98.6 ms on the 90%-factor partially sorted input. This is because it exits early when no swaps are needed, but it is still about 200 times slower than Quick Sort (1.2 ms). Insertion Sort is very slow on reversed data (113.7 ms) but is faster than almost every other algorithm on nearly-sorted inputs.

The table below summarizes the timing results for the large-array experiments.

Table 4: Elapsed time (ms) for $n = 10\,000$, averaged over 10 runs.

Algorithm	Random	Rev.	PS 10%	PS 30%	PS 50%	PS 70%	PS 90%
Bubble Sort	244.2	231.0	130.3	109.9	105.8	101.5	98.6
Quick Sort	1.2	0.9	0.8	1.1	0.9	1.2	1.3
Insertion Sort	56.6	113.7	13.2	5.1	3.4	2.1	1.9
Selection Sort	102.7	95.0	99.3	100.7	101.7	100.0	101.4
Merge Sort	2.8	1.9	2.1	2.0	1.8	1.9	1.9
Radix Sort	1.0	0.8	0.7	0.7	0.8	0.8	0.7

4.5 Results: Very Large Arrays ($n = 1\,000\,000$)

At one million elements, the difference between algorithms is very large. Bubble Sort takes almost 48 minutes on random data. Selection Sort takes about 15 minutes. In contrast, Radix Sort finishes in 91 ms on random data, Quick Sort in 207 ms, and Merge Sort in 293 ms.

Table 5: Best-performing algorithm on random input by size.

Input size	Best algorithm
10	Radix Sort
100	Quick Sort
1000	Quick Sort
10 000	Quick Sort
100 000	Radix Sort
1 000 000	Radix Sort

The next section visualizes the same results with plots, making it easier to compare algorithm behavior across input sizes and disorder levels.

4.6 Visual Analysis

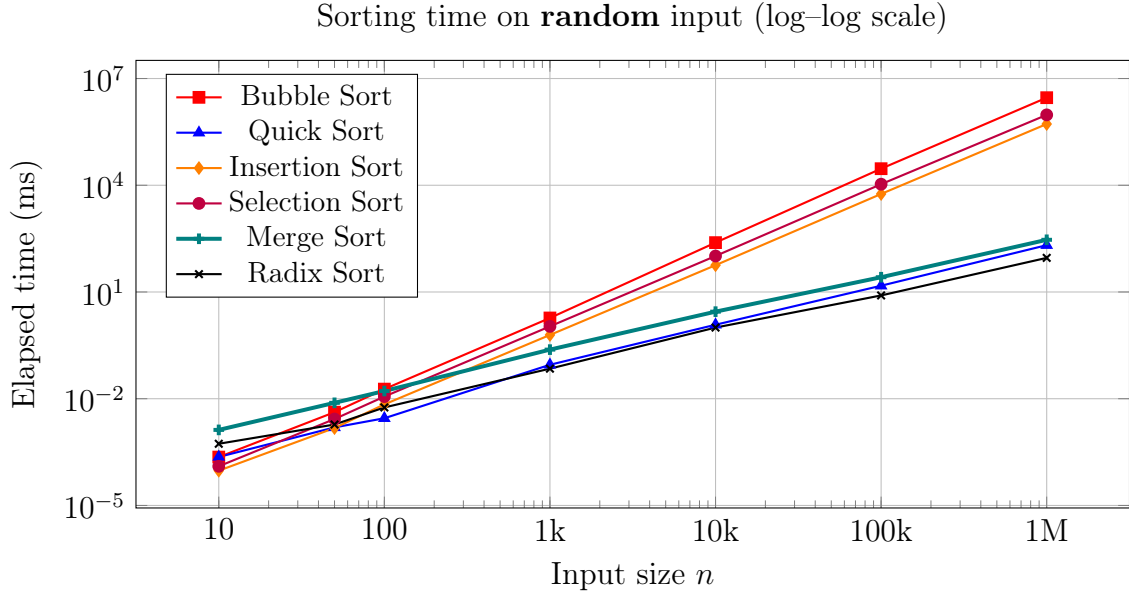


Figure 1: Performance on random arrays across all sizes (log-log scale). Markers are used so the curves can be distinguished even without color.

4.7 Key Observations

O1 – Insertion Sort is very sensitive to input order: On nearly-sorted data at $n = 1\,000\,000$ (90% factor), Insertion Sort takes 323 ms. This is close to Merge Sort (212 ms) and not far from Quick Sort (67 ms). On random data, the same algorithm takes 526 543 ms. This difference of about 1600 times shows how much the input order can change the speed of an algorithm.

O2 – Radix Sort grows slowly with size: When the input size increases 10 times (from $n = 100\,000$ to $n = 1\,000\,000$), Radix Sort on random data only increases about 11 times (from 8 ms to 91 ms). This matches its $O(dn)$ bound for a roughly constant number of digits d .

O3 – Quick Sort is reliable: With median-of-three pivot selection, Quick Sort handles reversed arrays well (139 ms at $n = 1\,000\,000$) and gives similar results across all

Effect of partial sorting on time at $n = 10\,000$

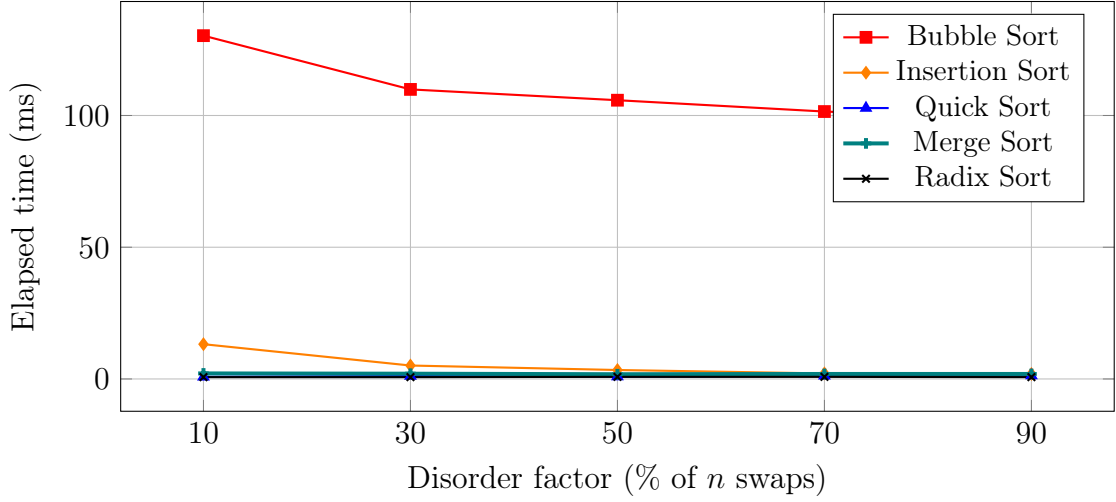


Figure 2: Impact of disorder level on sorting time at $n = 10\,000$. Marker shapes are used to distinguish algorithms without relying on color. Selection Sort omitted (flat at ≈ 100 ms).

input types. The slow $O(n^2)$ case is avoided.

O4 – Selection Sort always behaves the same: No matter what the input looks like, Selection Sort always does $\binom{n}{2}$ comparisons. This means its time barely changes across input types. In systems where consistent timing is more important than speed, this can be useful.

O5 – Merge Sort is slow on small inputs: At $n = 10$, Merge Sort is the slowest algorithm (0.001329 ms/sort) even though it has good asymptotic complexity. The reason is that creating and freeing helper arrays for each merge adds a lot of overhead at small sizes. A good fix would be to switch to Insertion Sort below a certain threshold.

5 Related Work

This section compares the present study with three relevant practical sources: McIlroy [4], Sedgewick and Wayne [5], and Bentley [1].

McIlroy’s optimistic sorting perspective. McIlroy [4] analyzes optimistic sorting techniques and how implementation choices such as pivot selection, partitioning strategy, and loop structure affect sort performance. His work is focused on practical, low-level tradeoffs in production-quality code.

The advantage of McIlroy over this paper is that he provides a deep engineering view of sorting implementations and explains how small changes can yield large real-world speedups.

The advantage of this paper over McIlroy is that it measures six different algorithms under a controlled partial-disorder model and reports timing results across seven input sizes up to 10^6 .

Sedgewick and Wayne’s practical algorithm analysis. Sedgewick and Wayne [5] offer a practical treatment of Quick Sort, Merge Sort, and Radix Sort, including empirical comparisons and guidance for implementation. Their text emphasizes how input characteristics and algorithm variants influence actual running time.

Their advantage is the broad survey of algorithmic variants and well-tested code patterns.

This paper’s advantage is the narrower focus on partially sorted inputs with parameterized disorder and the inclusion of simpler algorithms such as Bubble Sort and Insertion Sort.

Bentley’s nearly-sorted input work. Bentley [1] discusses why adaptive methods like Insertion Sort and hybrid approaches such as Timsort perform well on nearly-sorted data. His work is a classic reference for choosing sorting strategies in systems that frequently see partially ordered inputs.

The advantage of Bentley over this paper is that he covers adaptive hybrid algorithms and production use cases.

The advantage of this paper is that it supplies direct timing data for a range of disorder levels, showing when simple algorithms remain competitive.

Theoretical foundation. Knuth [3] and Cormen et al. [2] provide the rigorous theoretical background for sorting complexity and lower bounds.

Their advantage is the depth of formal analysis, but they do not focus on the empirical effects of controlled partial disorder.

This paper therefore complements them by presenting experimental evidence on how input ordering affects modern sorting performance.

6 Conclusions and Future Work

6.1 Summary

This paper compared six sorting algorithms — Bubble Sort, Insertion Sort, Selection Sort, Quick Sort, Merge Sort, and Radix Sort — using seven different input types and seven input sizes ranging over five orders of magnitude. The main findings are:

1. **For large general-purpose sorting**, Quick Sort and Radix Sort are the fastest. Radix Sort wins at $n = 10^6$ on random data (91 ms vs. 207 ms for Quick Sort).
2. **For nearly-sorted data**, Insertion Sort is as fast as $O(n \log n)$ algorithms and much faster at medium sizes.
3. **Merge Sort** gives the most stable results across all input types, which makes it a good choice when you need consistent performance.
4. **Selection Sort** is not affected by input order but is slow; it should not be used for $n \gtrsim 10^4$.
5. **Bubble Sort** is only suitable for very small inputs or for teaching purposes.

6.2 Problems Encountered

The main challenge was measuring time accurately for small array sizes. At $n = 10$, a single sort takes just a few nanoseconds, which is below what `clock()` can measure. This

was solved by running each test 1 000 000 times and averaging the result. At $n = 10^6$, each test was only run once, so there is more variation in the results, but the times are large enough that this does not matter much.

6.3 Future Work

This study successfully built the benchmark and collected results across seven sizes and seven input variants. It did not yet test other data types or different computer systems, which are planned improvements.

- **Cache profiling:** Using tools like `perf` or `Valgrind/Cachegrind` to count cache misses, especially when comparing Merge Sort and Quick Sort.
- **Hybrid algorithms:** Testing Timsort and Introsort, which switch between algorithms depending on the input, and comparing them to the current results.
- **Parallel versions:** Running parallel Merge Sort and parallel Radix Sort on multi-core machines to see how CPU count affects speed.
- **Other data types:** Testing the algorithms on floating-point numbers, text strings, and other data types to see whether the same relative ordering holds.
- **Cross-machine comparison:** Running the benchmark on other computers with different CPUs and memory configurations to identify which hardware factors influence the results most.

References

- [1] Jon L. Bentley. *Programming Pearls*. Addison-Wesley, 2nd edition, 1999.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4th edition, 2022.
- [3] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd edition, 1998.
- [4] M. Douglas McIlroy. Optimistic sorting. *Software: Practice and Experience*, 23(6):543–552, 1993.
- [5] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4th edition, 2011.

A Pseudocode

This appendix contains short, easy-to-understand pseudocode for the two main helper routines used in the benchmark.

A.1 Partially sorted array generator

```
1 function generatePartiallySortedArray(n, factor):  
2     A = [0, 1, 2, ..., n-1]  
3     swaps = n / factor  
4     for i from 1 to swaps:  
5         a = random index in [0, n-1]  
6         b = random index in [0, n-1]  
7         swap(A[a], A[b])  
8     return A
```

A.2 Median-of-three pivot selection

```
1 function choosePivot(A, low, high):  
2     mid = floor((low + high) / 2)  
3     sort the three values A[low], A[mid], A[high]  
4     return the median value
```